



KOLEGJI UNIVERSITAR UBT
Fakulteti Shkenca Kompjuterike dhe Inxhinieri

Testimi i Softuerit dhe Sigurimi i Cilësisë

Çështja qendrore të mirëmbajtjes

Mësimdhënësi:

prof.ass.Dr.sc.Liridon HOTI,

Student:

Rone Halili

232470625

Maj/2026
Prishtinë

PËRMBAJTJA

1.Hyrja dhe prezantimi i temës	6
1.1 Rëndësia e temës	6
1.2 Qëllimet e punimit	6
1.3 Pyetjet kërkimore	7
1.4 Konteksti	7
2. Përmbledhje teorike	7
2.1 Çfarë ndodh pas dorëzimit? (nga librat)	7
2.2 Katër llojet e mirëmbajtjes pas dorëzimit	8
2.3 Çështjet tipike të mirëmbajtjes pas dorëzimit	11
2.4 Testimi i softuerit dhe QA në fazën e mirëmbajtjes	11
2.5 Llojet dhe metodat e testimit	12
2.6 Çfarë duhet dorëzuar që mirëmbajtja të jetë e mundur?	13
2.7 Procesi i mirëmbajtjes (model i thjeshtuar)	13
3.Metodologjia e punimit	14
3.1 Qasja	15
3.2 Veglat	15
3.3 Kriteret e suksesit për një patch mirëmbajtjeje	16
4. Analiza praktike	17
4.1 Konteksti	17
4.2 Dorëzimi fillestar (v1.0)	18
4.3 Qështjet që u shfaqen pas dorëzimit	18
4.4 Mirëmbajtje korrigjuese — eksport CSV (pa BOM / me BOM)	19
4.5. Raporti i termineve sipas departamentit — mirëmbajtje perfekte	20
4.6.Përditësimi i librarive të sigurisë dhe smoke test i JWT (mirëmbajtje parandaluese / adaptuese)	21
4.7 Tabela test case (pas dorëzimit)	23

4.8 Defekte dhe metrika (version për Hospital-Menagment)	24
4.9 Diskutim i shkurtër i rezultateve	25
5. Diskutim dhe konkluzione	25
5.1 Lidhja me teori dhe librat	25
5.2 Mësimet kryesore	26
5.3 Sugjerime	26
5.4 Konkluzione	27
6. Referencat dhe burimet	27

LISTA E TABELAVE

Tabela 1.Llojet e mirëmbajtjes pas dorëzimit dhe lidhja me testimin	10
Tabela 2.Nivelet e testimit	12
Tabela 3.Veglat e përdorura në procesin e mirëmbajtjes	15
Tabela 4.Skema e test case-ve dhe rezultatet e verifikimit	23
Tabela 5.Defekte / probleme të identifikuara	24
Tabela 6.Metrikat e mirëmbajtjes para dhe pas ndreqjes	25

LISTA E FIGURAVE

Figura 1. Cikli i mirëmbajtjes së softuerit pas dorëzimit	14
Figura 2. Procesi iterativ i testimit pas çdo ndryshimi	16
Figura 3. Testi TC-CSV01 (pa BOM)	19
Figura 4.. Testi TC-CSV02 (me BOM)	20
Figura 5. Testimi i raportit para shtimit të endpoint-it (404)	20
Figura 6. Pas mirëmbajtjes perfekte: testi TC-RAP01	21
Figura 7. Analiza e dobësive të varësive — npm audit	22
Figura 8. Mirëmbajtje parandaluese — pas veprimi npm audit fix	22
Figura 9. Smoke test pas përditësimit të varësive	23

1.Hyrja dhe prezantimi i temës

1.1 Rëndësia e temës

Kur një projekt softueri **përfundon dhe dorëzohet** te klienti ose përdoruesi final, puna e ekipit të zhvillimit **nuk mbaron**. Fillon faza më e gjatë e ciklit të jetës: **mirëmbajtja** (ang. software maintenance). Sipas Pressman dhe Maxim (2019), pas dorëzimit sistemi hyn në përdorim real ku shfaqen defekte të paparashikuara, kërkesa për ndryshime ligjore, përditësime platformash dhe kërkesa të reja nga përdoruesit — të gjitha këto janë **qështje mirëmbajtjeje**.

Sommerville (2016) e përkufizon mirëmbajtjen si procesin e modifikimit të softuerit **pas dorëzimit** për të korrigjuar defekte, përmirësuar performancën ose atributet e tjera, ose për t'u përshtatur me mjedisin e ndryshuar. Kjo fazë shpesh zgjat më shumë se zhvillimi fillestar dhe konsumon pjesën më të madhe të kostos totale — prandaj në praktikë dhe në literaturë quhet problemi më i madh pas “përfundimit” të projektit.

Në lëndën Testimi i Softuerit dhe Sigurimi i Cilësisë, mirëmbajtja pas dorëzimit lidhet drejtpërdrejt me **testimin e regresionit**, menaxhimin e versioneve, dokumentacionin, SLA-të e mbështetjes dhe proceset QA që garantojnë se çdo ndryshim i vogël nuk prish sistemin që tashmë funksionon në prodhim.

1.2 Qëllimet e punimit

- 1.Të sqarojë çfarë ndodh me softuerin **pas dorëzimit** dhe cilat janë qështjet tipike të mirëmbajtjes.
- 2.Të paraqesë bazën teorike të testimit dhe QA-së në kontekstin e mirëmbajtjes (sipas literaturës së standardeve dhe librave të inxhinierisë së softuerit).
- 3.Të përshkruajë procesin dhe veglat për menaxhimin e ndryshimeve dhe testimit pas lëshimit.
- 4.Të analizojë një rast praktik (dorëzim → incident → patch → regresion).
- 5.Të propozojë masa për mirëmbajtje më të sigurt dhe më të lirë.

1.3 Pyetjet kërkimore

- Çfarë problemesh lindin menjëherë pas dorëzimit të një projekti softuerik?
- Si klasifikohen aktivitetet e mirëmbajtjes sipas standardeve dhe literaturës?
- Si ndihmon testimi (veçanërisht regresioni) në fazën pas dorëzimit?
- Çfarë duhet të dorëzohet bashkë me produktin që mirëmbajtja të jetë e mundur?

1.4 Konteksti

Puna teoriale dhe metodologjike bazohet në literaturën e mirëmbajtjes dhe testim (Pressman, 2019; Sommerville, 2016; ISO 14764) dhe në modelin e studimit të rastit të përdorur në literaturë (sistem web + API pas dorëzimit). **Analiza empirike** në kapitullin 4 aplikon të njëjtat kategori (korrigjuese, perfektive, parandaluese) dhe test case në **projektin real të lëndës** — sistemin e brendshëm të menaxhimit të spitalit (Hospital-Management) — i dorëzuar si version 1.0 dhe i testuar pas dorëzimit. Kërkesat, defektet, ndryshimet në kod dhe rezultatet e testeve u regjistruan gjatë testimit dhe mirëmbajtjesë organizuara për punimin seminarik.

2. Përmbledhje teorike

2.1 Çfarë ndodh pas dorëzimit? (nga librat)

Pressman dhe Maxim (2019) theksojnë se pas instalimit dhe dorëzimit, softueri hyn në fazën operacionale ku defektet dhe kërkesat për ndryshim shfaqen në kontekst real — jo vetëm në mjedisin e testit. Sommerville (2016, kapitulli mbi evolucionin e softuerit) e quan këtë **evolucion**: sistemi duhet të ndryshojë vazhdimisht për të mbetur i dobishëm, ndryshe vjetërohet.

ISO/IEC 14764 (2006) e përkufizon formalisht **mirëmbajtjen e softuerit** si:

“Aktivitetet e nevojshme për të ofruar mbështetje me kosto të optimizuar për një produkt softuerik, pasi ai është dorëzuar te klienti dhe po përdoret.”

2.2 Katër llojet e mirëmbajtjes pas dorëzimit

Mirëmbajtja korrigjuese (Corrective Maintenance)

Kjo formë e mirëmbajtjes realizohet kur pas dorëzimit të sistemit zbulohen defekte ose gabime funksionale gjatë përdorimit real nga përdoruesit. Sipas Sommerville (2016), shumica e ndërhyrjeve të para pas lëshimit lidhen me korrigjimin e gabimeve që nuk janë zbuluar gjatë zhvillimit ose testimit fillestar. Shembuj tipikë janë:

- mosruajtja e të dhënave,
- dështimi i login-it,
- gabimet në raportime,
- crash-et e aplikacionit.

Në këtë fazë, testimi ka rol kritik. Fillimisht krijohet një test që riprodhon defektin, pastaj implementohet korrigjimi dhe në fund realizohet regression testing për të verifikuar që ndryshimi nuk ka prishur funksione të tjera të sistemit. Në praktikë, mirëmbajtja korrigjuese është një nga aktivitetet më të zakonshme dhe më urgjente pas dorëzimit të softuerit.

Mirëmbajtja adaptuese (Adaptive Maintenance)

Mirëmbajtja adaptuese ndodh kur sistemi duhet të përshtatet me ndryshime të jashtme teknologjike ose operative. Këto ndryshime nuk lidhen drejtpërdrejt me gabime në kod, por me ndryshime në mjedisin ku softueri funksionon. Shembuj të zakonshëm përfshijnë:

- përditësimin e sistemit operativ,
- ndryshimet në databazë,
- versionet e reja të browserëve,
- API të reja,
- ndryshime në rregullore ose standarde sigurie.

Për shembull, një aplikacion mund të funksionojë në Java 17, por pas migrimit të serverit në Java 21 të kërkojë ndryshime shtesë për kompatibilitet. Në këtë lloj mirëmbajtjeje përdoren: • compatibility testing,

- smoke testing,
- integration testing.

Qëllimi është që sistemi të vazhdojë të funksionojë normalisht edhe pas ndryshimeve në mjedisin teknologjik.

Mirëmbajtja perfekte (Perfective Maintenance)

Mirëmbajtja perfekte fokusohet në përmirësimin e performancës, përdorshmërisë ose efikasitetit të sistemit edhe kur nuk ekziston ndonjë defekt konkret. Sipas Pressman dhe Maxim (2019), kërkesat për përmirësime shpesh vijnë nga përdoruesit pas përdorimit real të sistemit. Shembuj: • optimizimi i performancës, • përmirësimi i ndërfaqes grafike,

- shtimi i raporteve të reja,
- përmirësimi i kërimit,
- ulja e kohës së përgjigjes.

Ky lloj mirëmbajtjeje ndikon drejtpërdrejt në përvojën e përdoruesit dhe në cilësinë e perceptuar të softuerit.

Për verifikim përdoren:

- performance testing,
- usability testing,
- regression testing.

Megjithëse nuk korrigjon gabime, mirëmbajtja perfekte rrit vlerën dhe jetëgjatësinë e sistemit.

Mirëmbajtja parandaluese (Preventive Maintenance)

Mirëmbajtja parandaluese realizohet për të ulur rrezikun e problemeve të ardhshme dhe për ta bërë sistemin më të lehtë për mirëmbajtje në të ardhmen.

Kjo përfshin:

- refaktorimin e kodit,
- përditësimin e librarive,
- eliminimin e borxhit teknik,
- përmirësimin e dokumentacionit,

- rritjen e coverage të testeve. Një shembull i zakonshëm është përditësimi i një librerie që ka vulnerabilitete sigurie (CVE), edhe pse sistemi ende funksionon normalisht.

Në këtë fazë përdoren:

- static code analysis,
- code review,
- automated testing,
- tools si SonarQube ose JaCoCo.

Mirëmbajtja parandaluese ndihmon në reduktimin e kostove afatgjata dhe rrit stabilitetin e sistemit. Shumë organizata moderne investojnë në këtë lloj mirëmbajtjeje për të shmangur probleme kritike në të ardhmen.

Sipas ISO/IEC 14764 dhe përputhjes me Pressman (2019) dhe Sommerville (2016):

Lloji	Kur ndodh pas dorëzimit	Shembull	Lidhja me testimin
Korrigjuese (corrective)	Përdoruesi raporton bug	Forma nuk ruan të dhëna	Test që riprodhon bug-un; test regresioni pas fix-it
Adaptuese (adaptive)	Ndryshon mjedisi	Serveri Linux përditësohet; API e re e shfletuesit	Test kompatibiliteti, smoke test në mjedis të ri
Perfektive (perfective)	Përmirësim pa “bug”	Raport më i shpejtë, UI më i qartë	Test performancë; regresion funksional
Parandaluese (preventive)	UI rrezikun e ardhshëm	Refaktorim, përditësim librarish sigurie	Rritje coverage; analizë statike (SonarQube)

Tabela 1. Llojet e mirëmbajtjes pas dorëzimit dhe lidhja me testimin

2.3 Çështjet tipike të mirëmbajtjes pas dorëzimit

Nga sinteza e literaturës (Pressman, 2019; Sommerville, 2016; Myers et al., 2011),

pas dorëzimit të softuerit shfaqen disa çështje tipike që ndikojnë në procesin e mirëmbajtjes dhe cilësinë e sistemit në prodhim:

Mungesa e dokumentacionit — Ekipi i mirëmbajtjes shpesh nuk e njeh mirë sistemin për shkak të dokumentacionit të pamjaftueshëm ose jo të plotë, duke e bërë kodin të vështirë për t'u kuptuar ("kod i huaj"). Kjo e ngadalëson analizën e ndryshimeve dhe riparimin e defekteve.

Borxhi teknik — Zgjidhjet e shpejta gjatë zhvillimit krijojnë struktura jo optimale në kod, të cilat më vonë e komplikojnë mirëmbajtjen dhe ndryshimet e reja.

Mungesa e testeve automatike — Pa teste automatike, çdo ndryshim kërkon testim manual të gjithë sistemit, gjë që e rrit kohën, koston dhe rrezikun e gabimeve. Kjo gjithashtu e vështirëson testimin e regresionit.

Menaxhimi i kërkesave — Pas dorëzimit, përdoruesit shpesh kërkojnë ndryshime të vogla që në fakt janë funksione të reja, duke e bërë procesin e mirëmbajtjes më kompleks dhe më të vazhdueshëm.

Versionimi dhe konfigurimi — Menaxhimi i versioneve të ndryshme (prodhim, test) dhe shpërndarja e patch-eve të klientëve të ndryshëm mund të krijojë inkonsistenca dhe probleme në funksionim.

Ekipi dhe njerëzit — Largimi i zhvilluesve që kanë ndërtuar sistemin sjell humbje të njohurive kritike, duke e bërë mirëmbajtjen më të vështirë dhe më të ngadaltë.

Kosto dhe SLA (Service Level Agreements) — Mirëmbajtja është e kufizuar nga kohët dhe kostot e reagimit ndaj incidenteve, duke krijuar presion për ndryshime të shpejta që duhet të testohen me kujdes për të shmangur defektet në prodhim.

Në përgjithësi, këto çështje tregojnë se mirëmbajtja pas dorëzimit nuk është vetëm proces teknik, por kombinim i faktorëve teknikë, organizativë dhe të procesit të zhvillimit, dhe lidhet ngushtë me nevojën për testim të vazhdueshëm dhe sigurim të cilësisë së sistemit.

2.4 Testimi i softuerit dhe QA në fazën e mirëmbajtjes

Testimi (Myers et al., 2011) pas dorëzimit fokusohet te:

- **Verifikimi i korrigjimeve** — defekti raportuar nuk përsëritet.

- **Testimi i regresionit** — funksionaliteti i vjetër nuk priset nga fix-i ose nga funksioni i ri (Myers et al., 2011, diskutojnë regresionin si rrezik kryesor pas ndryshimeve).
- **Testimi i pranimit të mirëmbajtjes** — klienti konfirmon se patch-i zgjidh problemin.

Sigurimi i cilësisë (QA) në mirëmbajtje përfshin: proces për raportim defektesh, prioritet (kritik/major/minor), release management, CI që ekzekuton testet para çdo patch-i, dhe auditim periodik i cilësisë (ISO/IEC/IEEE 29119).

Këto aktivitete janë thelbësore për të siguruar që çdo ndryshim pas dorëzimit të mos ndikojë negativisht në stabilitetin ekzistues të sistemit.

2.5 Llojet dhe metodat e testimit

Pas dorëzimit të softuerit, testimi realizohet në disa nivele për të siguruar që ndryshimet (bug fixes ose funksionalitete të reja) nuk ndikojnë në stabilitetin ekzistues të sistemit. Këto nivele të testimit janë të ndërlidhura ngushtë me procesin e mirëmbajtjes dhe regresionit.

Lloji	Përdorimi pas dorëzimit
Unit	Teston klasën e ndryshuar para se të shkojë në prodhim
Integrimi	API + bazë të dhënash pas ndryshimit
Sistemi / E2E	Rrjedha e përdoruesit (login → raport)
Pranimi	Klienti/professori konfirmon se kërkesa e mirëmbajtjes u plotësua

Tabela 2. Nivelet e testimit

Në kontekst të mirëmbajtjes, këto nivele ndihmojnë në identifikimin e hershëm të defekteve dhe reduktimin e regresioneve pas çdo ndryshimi.

Gjithashtu, përdoren teknika të ndryshme të testimit:

- Black-box testing: EP dhe BVA për fusha të reja (p.sh. datë, numër kreditesh)
- White-box testing: coverage i kodit të prekur nga patch-i

Automatizimi i testimit me vegla si JUnit, Selenium dhe Postman është thelbësor për ekzekutimin e shpejtë të testimeve të regresionit dhe për rritjen e efikasitetit të procesit të mirëmbajtjes (Pressman, 2019).

2.6 Çfarë duhet dorëzuar që mirëmbajtja të jetë e mundur?

Sipas Sommerville (2016) dhe standardeve IEEE 829 dhe ISO/IEC/IEEE 29119-3, procesi i dorëzimit të softuerit (handover) nuk kufizohet vetëm në aplikacionin final, por përfshin një paketë të plotë artefaktesh që mundësojnë mirëmbajtje efikase dhe të sigurt pas lëshimit. Kjo paketë është thelbësore për të siguruar vazhdimësinë e zhvillimit dhe testimit në të ardhmen.

Paketa e dorëzimit përfshin:

- Kodi burimor dhe skriptet e build-it
- Dokumentacioni i sistemit dhe i përdoruesit
- Modeli i bazës së të dhënave dhe skriptet e migrimit
- Suita e testeve (unit, integrim) dhe udhëzime ekzekutimi
- Konfigurimi i mjediseve (dev, test, prod)
- Lista e defekteve të njohura dhe kufizimeve

Mungesa e këtyre elementeve e bën procesin e mirëmbajtjes të vështirë dhe të pasigurt, duke e kthyer atë në një proces të ngjashëm me “riparim në errësirë”, ku çdo ndryshim kërkon analizë të thellë pa mbështetje adekuate dokumentacioni apo testimi.

2.7 Procesi i mirëmbajtjes (model i thjeshtuar)

Bazuar në ISO/IEC 14764 dhe Pressman (2019), procesi i mirëmbajtjes së softuerit ndjek një cikël të strukturuar aktivitetesh që fillon nga raportimi i problemit dhe përfundon me lëshimin dhe dokumentimin e ndryshimeve. Ky proces është i rëndësishëm për të siguruar që çdo ndryshim në sistem të jetë i kontrolluar, i testuar dhe i gjurmueshëm.

Procesi i mirëmbajtjes përfshin këto hapa:

1. Raportim problemi (helpdesk, email, GitHub Issues)
2. Analizë dhe klasifikim (bug vs. kërkesë e re; severitet)

3. Planifikim ndryshimi (versioni 1.0.1, 1.1.0)
4. Implementim (fix, refaktorim)
5. Testim (fix verification + regresion)
6. Lëshim patch-i (deploy, rollback nëse dështon)
7. Mbyllje dhe dokumentim (release notes)

Ky cikël siguron që çdo ndryshim të kalojë përmes një procesi të kontrolluar dhe të verifikuar, duke reduktuar rrezikun e defekteve në prodhim dhe duke ruajtur stabilitetin e sistemit gjatë evolucionit të tij.

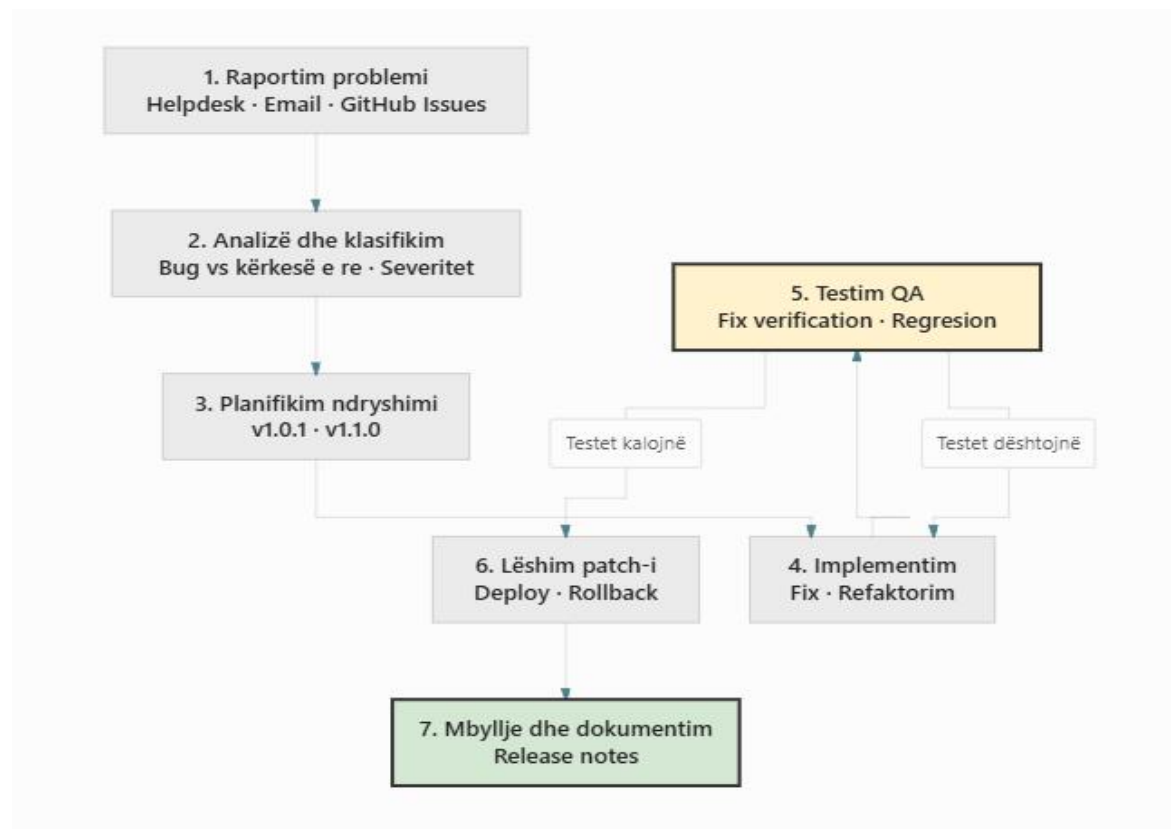


Figura 1. Cikli i mirëmbajtjes së softuerit pas dorëzimit

3. Metodologjia e punimit

Puna ndahet në dy shtresa: teoria dhe metodologjia bazohen në literaturën e mirëmbajtjes dhe testim (Pressman, 2019; Sommerville, 2016; ISO 14764) dhe në modelin e studimit të rastit (sistem web + API pas dorëzimit). Analiza praktike në kapitullin 4 aplikon të njëjtat kategori mirëmbajtjeje dhe test case në projektin real të lëndës — sistemin e menaxhimit të spitalit (Hospital-Management) — me incidente, ndryshime në kod dhe rezultate testimi të regjistruara pas dorëzimit të versionit 1.0.

3.1 Qasja

U ndoq një **studim rasti** mbi sistemin e menaxhimit të spitalit (aplikacion web + API REST), i dorëzua si version 1.0 .Metodologjia ndjek standardin **ISO/IEC/IEEE 29119** (planifikim, dizajn test case, ekzekutim, raportim) dhe procesin e mirëmbajtjes së përshkruar në kapitullin 2.

Struktura e analizës në kapitullin 4 pasqyron modelin e literaturës (dorëzim → problem → ndryshim → verifikim / regresion), por të dhënat dhe screenshot-et vijnë nga **projekti real**, jo nga shembulli akademik i studentëve.

3.2 Veglat

Në këtë punim janë përdorur disa vegla.

Vegla	Roli në mirëmbajtje
Git / GitHub	Versionim, branch
npm / npm audit	Varësi, përditësim sigurie
Node.js / Express	API pas ndryshimeve
MySQL Workbench	Të dhëna testuese
PowerShell	Test API me JWT
React	UI (CSV, takime)

Tabela 3. Veglat e përdorura në procesin e mirëmbajtjes

Përdorimi i këtyre veglave siguron një proces të strukturuar të zhvillimit dhe mirëmbajtjes, duke reduktuar rrezikun e defekteve në prodhim dhe duke përmirësuar cilësinë e përgjithshme të sistemit përmes testimit të vazhdueshëm dhe versionimit të kontrolluar.

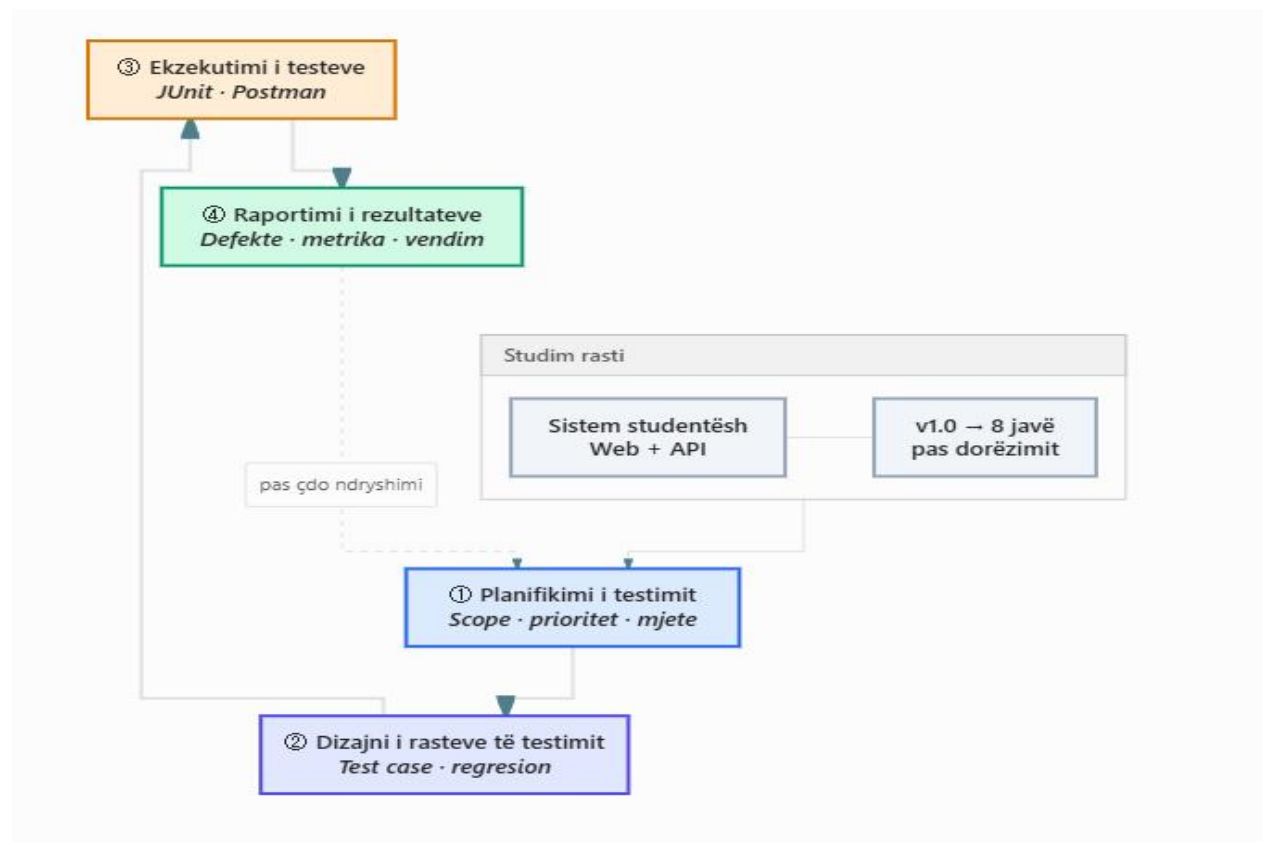


Figura 2. Procesi iterativ i testimit pas çdo ndryshimi

3.3 Kriteret e suksesit për një patch mirëmbajtjeje

Suksesi i një patch-i në procesin e mirëmbajtjes nuk vlerësohet vetëm nga implementimi i ndryshimit, por nga verifikimi i plotë i tij përmes testimit dhe validimit në mjedisin e sistemit. Për këtë arsye, përcaktohen disa kriteret kryesore që sigurojnë që ndryshimi është i qëndrueshëm dhe nuk krijon probleme të reja.

Një patch konsiderohet i suksesshëm nëse plotësohen këto kushte:

- Defekti i raportuar duhet të jetë i riprodhuar dhe i verifikuar përmes një testi specifik para se të aplikohet zgjidhja.

- Implementimi i fix-it duhet të kalojë me sukses testin e ri si dhe të gjithë testet e regresionit ekzistues.
- Të gjitha ndryshimet duhet të dokumentohen në formë të release notes për të siguruar gjurmueshmëri.
- Pas deploy-it, sistemi duhet të funksionojë stabilisht pa defekte kritike të reja gjatë një periudhe fillestare monitorimi (p.sh. 48 orë).
- Pipeline i testimit (ku ekziston) duhet të ekzekutohet me sukses para çdo lëshimi në prodhim.

Në këtë mënyrë, çdo patch kalon përmes një procesi të kontrolluar verifikimi që minimizon rrezikun e regresioneve dhe siguron stabilitetin e sistemit në mjedisin real të përdorimit.

4. Analiza praktike

Kapitulli 4 ndjek strukturën e analizës së mirëmbajtjes pas dorëzimit të propozuar në literaturë dhe në udhëzimet e lëndës (identifikim problemi, klasifikim, implementim ndryshimi, verifikim dhe regresion). Ndryshe nga shembujt e përgjithshëm të tekstit mësimor, këtu çdo aktivitet dokumentohet në **sistemin e menaxhimit të spitalit** me test case TC-M01–M05 dhe figura përkatëse

4.1 Konteksti

Shembulli praktik është **sistemi i brendshëm i menaxhimit të spitalit** (aplikacion web + API REST), i përdorur nga personeli i spitalit për regjistrimin e pacientëve, terminet dhe stafin mjekësor.

Sistemi u dorëzua si version 1.0 në fund të semestrit dhe u **monitorua gjatë 8 javëve** pas dorëzimit. Në këtë fazë u shfaqën kërkesa për ndryshime, defekte në përdorim real dhe nevojë për përditësime teknike — domosdoshmërisht **mirëmbajtje**, jo “fund i punës”.

Gjatë monitorimit u regjistruan **kërkesat, defektet, ndryshimet në kod dhe rezultatet e testeve** pas çdo përditësimi, për të analizuar rolin e **testimit dhe QA** (veçanërisht regresionit) në mirëmbajtje. **Analiza e plotë** e këtij rasti gjendet në kapitullin 4.

4.2 Dorëzimi fillestar (v1.0)

Rasti i analizuar është sistemi i brendshëm i menaxhimit të spitalit (aplikacion web me backend REST API dhe bazë të dhënash relacionale), i zhvilluar si projekt semestri dhe i dorëzuar në versionin 1.0 në fund të semestrit.

Sistemi përdoret nga personeli administrativ dhe mjekësor (jo nga pacientët e jashtëm) për menaxhimin e të dhënave bazë të spitalit në mjedis të mbyllur (rrjet lokal / intranet).

Funksionalitetet e v1.0:

- Autentifikim: hyrje përdoruesi sipas rolit (p.sh. administrator, infermier, reception);
- Menaxhim pacientësh: regjistrim pacienti, listim, kërkim sipas emrit ose numrit të kartelës;
- Termine / vizita: krijim dhe listim i termineve (pacient, data, ora, mjeku);
- Personel: listim i stafit mjekësor (emër, specializim, departament);
- Eksport i thjeshtë: eksport listës së pacientëve ose termineve në CSV (për arkiv administrativ).

4.3 Çështjet që u shfaqën pas dorëzimit

Pas dorëzimit të versionit 1.0, sistemi i menaxhimit të spitalit filloi të përdorej në mënyrë më intensive nga personeli administrativ dhe mjekësor në mjedisin testues të spitalit. Gjatë përdorimit real u identifikuan disa probleme dhe kërkesa të reja që nuk ishin vërejtur gjatë fazës së zhvillimit dhe testimit fillestar. Këto çështje lidhen me mënyrën e shfaqjes së të dhënave, mungesën e disa funksioneve raportuese dhe nevojën për përditësime të sigurisë në backend.

Një pjesë e problemeve u klasifikuan si mirëmbajtje korrigjuese, sepse lidhen me defekte që ndikonin drejtpërdrejt në përdorimin e sistemit, si problemi me encoding-un e eksportit CSV në Excel. Disa kërkesa të tjera u trajtuan si mirëmbajtje perfekte, pasi përdoruesit kërkuan funksionalitete shtesë për raportim dhe menaxhim më të mirë të të dhënave. Gjithashtu, u realizuan aktivitete parandaluese dhe adaptuese për përditësimin e librarive të sigurisë dhe verifikimin e autentifikimit JWT pas ndryshimeve në varësi.

Të gjitha incidentet dhe ndryshimet u dokumentuan përmes test case-ve dhe smoke test-eve pas çdo ndërhyrjeje. Në nënkapitujt vijues paraqiten rastet kryesore të mirëmbajtjes së realizuar në sistem, mënyra e testimit dhe rezultatet e verifikimit pas ndryshimeve.

4.4 Mirëmbajtje korigjuese — eksport CSV (pa BOM / me BOM)

Në kapitullin praktik kam testuar një problem tipik **pas dorëzimit** të sistemit tonë të menaxhimit të spitalit: **eksporti i të dhënave në CSV** dhe hapja e tyre në **Microsoft Excel**.

Kemi dy butona eksporti: një **pa BOM** (Byte Order Mark) dhe një **me BOM**. Të dyja skedarët janë UTF-8, por Excel në Windows shpesh **nuk e njeh UTF-8 pa BOM** dhe i lexon si encoding tjetër. Prandaj shkronjat shqipe **ë, ç, ü** dalin gabim — p.sh. *Gëzim* bëhet *GÃ«zim*, *Çlirim* bëhet *Ãlirim*.

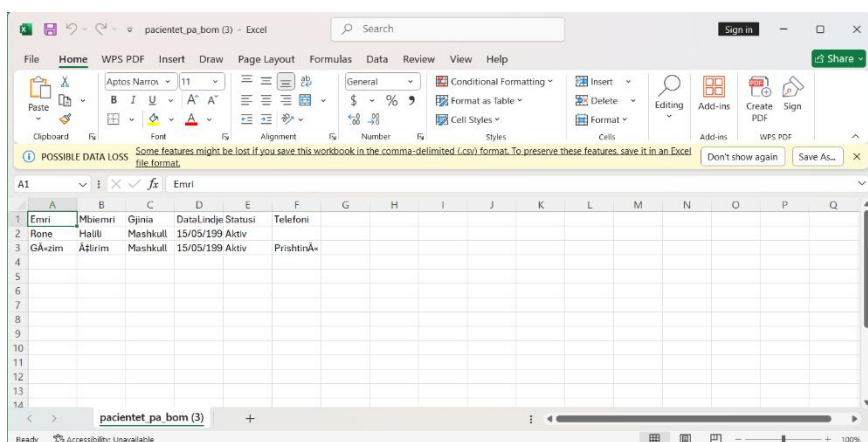


Figura 3. Testi TC-CSV01 (pa BOM)

Testi TC-CSV01 (pa BOM): eksportova listën e pacientëve dhe e hapa CSV-në në Excel.

Rezultati ishte **dështim** nga këndvështrimi i përdoruesit: të dhënat duken të prishura, edhe pse në databazë dhe në aplikacion janë të sakta. Kjo është **defekt i shfaqjes** pas mirëmbajtjes/përdorimit, jo gabim logjike në server.

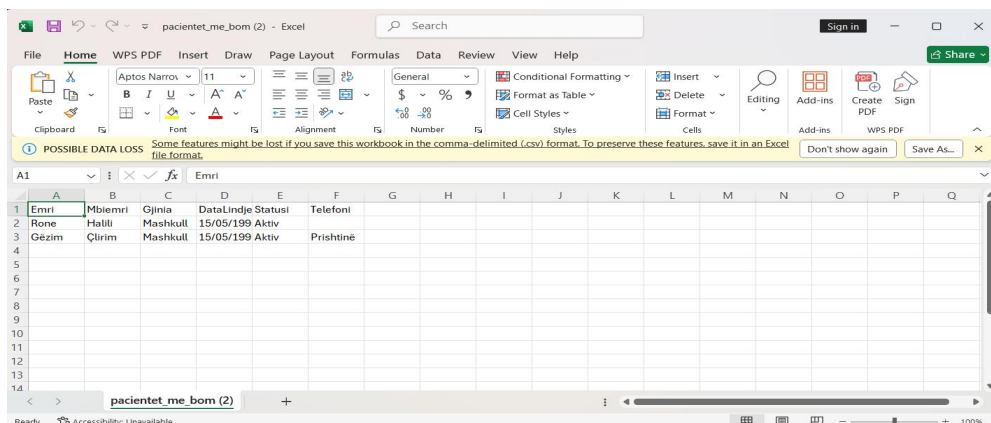


Figura 4. Testi TC-CSV02 (me BOM)

Testi TC-CSV02 (me BOM): e njëjta listë, por skedari fillon me **BOM UTF-8** (\uFEFF). Excel e njeh encoding-un dhe **karakteret shqipe shfaqen saktë**. Rezultati: **kalon** testin e pranueshmërisë për përdoruesin.

4.5. Raporti i termineve sipas departamentit — mirëmbajtje perfekte

```
PS C:\Windows\System32> Invoke-RestMethod -Method GET http://localhost:8000/takinet/raport/sipas-departamentit/1
Cannot GET /takinet/raport/sipas-departamentit/1 - 404 Not Found.
PS C:\Windows\System32> _
```

Figura 5. Testimi i raportit para shtimit të endpoint-it (404)


```
PS C:\Projects\Hospital-Menagment> npm audit
hub.com/advisories/GHSA-vghf-hv5q-vc2g
fix available via `npm audit fix`
node_modules/validator
  express-validator 0.2.0 - 6.4.1 || 7.1.0 - 7.2.1
  Depends on vulnerable versions of validator
  node_modules/express-validator

16 vulnerabilities (6 moderate, 10 high)

To address all issues, run:
  npm audit fix
PS C:\Projects\Hospital-Menagment>
```

Figura 7. Analiza e dobësive të varësive — npm audit

Komanda npm audit raportoi **16 dobësi** (6 moderate, 10 high), përfshirë varësi të lidhura me validator përmes express-validator (Figura 7). Pas veprimit npm audit fix, numri u reduktua në **3 dobësi** (1 moderate, 2 high), kryesisht në zinxhirin e paketës tar (Figura 8).

Kjo tregon se përditësimi automatik ka efekt të kufizuar dhe një pjesë e riskut mbetet për trajtim manual në fazat e ardhshme.

```
PS C:\Projects\Hospital-Menagment> npm audit fix
b.com/advisories/GHSA-r6q2-hw4h-h46w
fix available via `npm audit fix`
node_modules/tar
  @mapbox/node-pre-gyp <=1.0.11
  Depends on vulnerable versions of tar
  node_modules/@mapbox/node-pre-gyp

3 vulnerabilities (1 moderate, 2 high)

To address all issues, run:
  npm audit fix
PS C:\Projects\Hospital-Menagment>
```

Figura 8. Mirëmbajtje parandaluese — pas veprimit npm audit fix

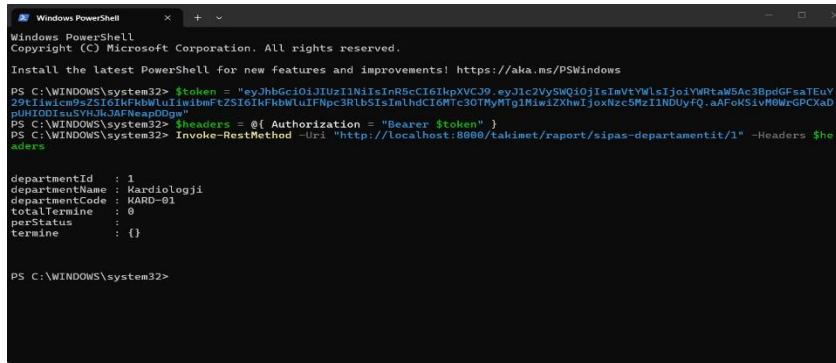


Figura 9.Smoke test pas përditësimit të varësive

Si **smoke test** pas përditësimit u verifikua: (1) hyrja në sistem si përdorues Admin; (2) thirrja e endpoint-it të mbrojtur GET /takimet/raport/sipas-departamentit/1 me header Authorization: Bearer <token>. Përgjigja përfshin departmentName: Kardiologji, departmentCode: KARD-01 dhe totalTermine: 0 (Figura 9), që konfirmon se mekanizmi JWT dhe API-ja e raportit **funksionojnë** pas audit fix. Vlera zero e termineve lidhet me mungesën e të dhënave testuese në databazë, jo me dështim të shërbimit.

4.7 Tabela test case (pas dorëzimit)

ID	Qëllimi	Hyrja	Pritet	Rezultati
TC-M01	Login	Admin, faqja 3000	Token aktiv	Kaluar
TC-M02	CSV pa BOM	Eksport → Excel	ë/ç prishen	Dështuar
TC-M03	CSV me BOM	Eksport → Excel	ë/ç OK	Kaluar
TC-M04	Raport dept.	GET raport/1 + token	JSON 200	Kaluar
TC-M05	Audit npm	audit → fix → login	JWT + API OK	Kaluar

Tabela 4. Skema e test case-ve dhe rezultatet e verifikimit

TC-M02 dhe TC-M03 testojnë mirëmbajtjen korrigjuese (encoding CSV/BOM). TC-M04 është mirëmbajtje perfekte (raport sipas departamentit). TC-M05 është parandaluese (npm audit dhe smoke test JWT).

4.8 Defekte dhe metrika (version për Hospital-Management)

ID		Përshkrimi	Shënim
<i>BUG-R1</i>		Encoding CSV pa BOM në Excel (ë, ç prishen)	Mirëmbajtje korrigjuese — shtim eksport me BOM
BUG-R2		Dropdown “Koha e lirë” bosh (mungesë rreshtash në Kohetelira)	Mungesë të dhënash / BVA — testim konfigurimi DB
BUG-R3		Pacient i lidhur me llogari Admin në listë	Konfigurim i gabuar Patients.userId
GAP-P1		Mungon raport terminesh sipas departamentit	Jo bug, por mungesë funksioni → perfekte (endpoint i ri)

Tabela 5. Defekte / probleme të identifikuara

Metrika	Gjendja fillestare (para ndreqjes)	Pas ndreqjes / testim
Test case manuale (TC-M01...M05)	—	5 të dokumentuara
Teste automatike	0	0 (nëse nuk ke Jest — shkruaj singlerisht)
Dobësi npm (npm audit)	16	3
Incidente kritike në testim	0	0
Funksione të reja (perfektive)	0	1 (raport dept.)

Ndreqje korrigjuese të verifikuara	0	1 (CSV BOM)
------------------------------------	---	-------------

Tabela 6.Metrikat e mirëmbajtjes para dhe pas ndreqjes

4.9 Diskutim i shkurtër i rezultateve

Çfarë funksionoi: Testimi manual i strukturuar përmes test case-ve (TC-M01–M05) lejoi identifikimin e problemeve reale pas dorëzimit: encoding-i i CSV-së në Excel (pa/me BOM), funksionimi i endpoint-it të ri për raportin sipas departamentit (Kardiologji), si dhe vazhdimësinë e autentifikimit JWT pas npm audit fix. Përditësimi i varësive (npm audit) reduktoi dobësitë nga 16 në 3, dhe smoke test-i (login + API e mbrojtur) konfirmoi që sistemi mbeti operativ. Dokumentimi me figura (para/pas) e bëri të qartë ndryshimin midis mirëmbajtjes korrigjuese, perfektive dhe parandaluese.

Çfarë nuk funksionoi / kufizime: Projekti nuk dispononte suitë testesh automatike (unit/integration); kjo shtoi varësinë nga testimi manual dhe konfigurimi i databazës (pacient, mjek, Kohetelira, staff), ku u humb kohë në diagnostikim. npm audit fix nuk eliminoi të gjitha dobësitë (mbeten 3). Raporti u testua me totalTermine: 0, që tregon nevojë për të dhëna testuese më të plota. Mungesa e dokumentimit të qartë të API-së në fazën fillestare e vështëroi testimin e funksioneve të reja.

Lidhja me literaturën: Kjo përputhet me Pressman (2019): kostoja e mirëmbajtjes rritet kur cilësia fillestare (testim, dokumentim, të dhëna konfigurimi) është e dobët dhe ndryshimet verifikohen kryesisht manualisht. Në fazat e ardhshme, automatizimi i testeve për login, API kritike dhe eksport CSV do të ulte rrezikun e regresioneve.

5. Diskutim dhe konkluzione

5.1 Lidhja me teori dhe librat

Puna praktike konfirmon parimet e Sommerville (2016): softueri evolvon pas dorëzimit; pa proces, testim dhe dokumentim, çdo ndryshim (edhe i vogël) mund të krijojë probleme të reja ose të zbulojë mangësi të vjetra. Në projektin e menaxhimit të spitalit u pa qartë kjo: sistemi “punonte” në përdorim, por pas dorëzimit u deshën ndërhyrje për eksport CSV, raport sipas departamentit dhe përditësim varësish.

Myers et al. (2011) ilustrojnë rëndësinë e regresionit dhe testimit të bazuar në rrezik: defekti i encoding-ut në Excel (BUG-R1) nuk ishte gabim logjike në server, por dështim i shfaqjes që përdoruesi e konsideron defekt; pa test pas ndryshimit, problemi do të mbetej i padukshëm në UI të brendshme. Testet TC-M01–M05 funksionuan si smoke dhe regresion manual për login, API dhe eksport.

ISO/IEC 14764 klasifikon saktë llojet e punës së kryer: korrigjuese (CSV/BOM), perfektive (endpoint raporti), parandaluese/adaptuese (npm audit). Kjo e lidh praktikën me terminologjinë standarde të mirëmbajtjes.

Pressman (2019) thekson koston e mirëmbajtjes: kur cilësia fillestare (teste automatike, dokumentacion API, të dhëna testuese në DB) është e ulët, koha shkon në diagnostikim manual — si te dropdown-et e takimeve dhe konfigurimi Staff / Kohetelira.

Përfundim i lidhjes: QA nuk është “shtesë” pas dorëzimit, por kusht që mirëmbajtja të mos ulë besimin e përdoruesit (encoding i saktë, raporte të sakta, siguri e varësive).

5.2 Mësimet kryesore

Dorëzimi i projektit $\neq$ fund i punës — duhet plan mirëmbajtjeje (çfarë ndryshon, si testohet, kush miraton).

Testet dhe skenarët dokumentohen edhe për të ardhmen — TC-M01–M05 ishin investim që sqaron çfarë u provua; pa to, diskutimi bëhet subjektiv.

Çdo ndryshim kalon verifikim — pas BOM, pas endpoint-it të ri, pas npm audit fix u bë smoke test (login + API); kjo përputhet me Pressman dhe Myers për regresion.

Problemet janë teknike dhe organizative — mungesa e të dhënave në DB (mjek, orë e lirë) dhe mungesa e testeve automatike janë po aq të rëndësishme sa bug-u i CSV-së.

Mirëmbajtja perfektive vjen nga kërkesa reale — raporti sipas departamentit nuk ishte “bug”, por nevojë e re e menaxhimit; duhet trajtuar ndryshe nga korrigjimi i encoding-ut.

5.3 Sugjerime

Gjatë zhvillimit (para dorëzimit):

Paketë dorëzimi: README deploy, skema DB, shembuj API (Postman/cURL), të dhëna testuese (patient, mjek, orare).

Teste automatike minimale: login JWT, 1–2 endpoint kritikë, eksport CSV (encoding).

Rregull për UTF-8/BOM në eksporte që hapen në Excel.

Pas dorëzimit:

Versionim semantik (p.sh. 1.0.1 patch për BOM, 1.1.0 minor për raport departamenti).

CI me npm test / npm audit (stack Node.js — jo vetëm mvn test nëse projekti është Express).

Log i ndryshimeve: lloji i mirëmbajtjes (korrigjuese / perfektive / parandaluese), TC-ID, rezultati.

Retrospektivë periodike: çfarë lloj pune dominoi (defekte shfaqjeje vs funksione të reja vs siguri).

Për ekipin Hospital-Management:

Dropdown-e të lidhura me DB të dokumentuara (pacient–user, mjek–staff–departament).

Raporte të testuara me të dhëna jo-zero për validim biznesi.

5.4 Konkluzione

Çështjet qendrore të mirëmbajtjes pas dorëzimit — korrigjimi i defekteve, përshtatja (varësi, mjedis), përmirësimet e kërkuara dhe parandalimi — u manifestuan edhe në një sistem real menaxhimi spitali. Analiza praktike tregoi se pa testim dhe QA, kostoja dhe rreziku rriten: nga encoding i CSV-së te mungesa e funksioneve raportuese dhe dobësitë e librarive.

Literatura (Pressman, 2019; Sommerville, 2016; ISO 14764; Myers et al., 2011) dhe rezultatet e testeve TC-M01–M05 mbështesin të njëjtën përfundim: mirëmbajtja e suksesshme fillon para dorëzimit (cilësi, dokumentacion, teste) dhe vazhdon pas tij me proces të disiplinuar. Në absence të testeve automatike, regresioni manual dhe smoke test mbeten të nevojshëm, por më të shtrenjtë dhe më të prirur ndaj gabimeve.

Për projektin e ekzaminuar, “projekti i dorëzuar” nuk është fund; është pikënisje e evolucionit. Me paketë mirëmbajtjeje, versionim dhe CI, ekipi mund të ulë rrezikun që ndryshimet e ardhshme — si ato të dokumentuara në këtë punim — të përsëriten pa kontroll.

6. Referencat dhe burimet

Auth0. (2024). node-jsonwebtoken . GitHub. <https://github.com/auth0/node-jsonwebtoken>

Ekipi Hospital-Management. (2025). Repozitori dhe dokumentacioni i brendshëm i projektit (version 1.0 dhe patch-e mirëmbajtjeje). [Projekt semestri, Kolegji Universitar UBT.]

GitHub, Inc. (2025). npm audit. <https://docs.npmjs.com/cli/v10/commands/npm-audit>

IEEE. (2008). IEEE Std 829-2008: IEEE Standard for Software and System Test Documentation. IEEE.

ISO/IEC 14764:2006. (2006). Software engineering — Software life cycle processes — Maintenance. International Organization for Standardization.

ISO/IEC/IEEE 29119-1:2022. (2022). Software and systems engineering — Software testing — Part 1: General concepts. ISO/IEC/IEEE.

Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley.

NIST. (2024). Secure Software Development Framework (SSDF), SP 800-218. National Institute of Standards and Technology. <https://csrc.nist.gov/projects/ssdf>

OWASP Foundation. (2021). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

